Documento de Arquitectura - Software III

Stefania Herrera Ochoa
C.C. 1090274506
Allison Lopez Caranton
C.C 1088827979
Daniel Valencia Arenas
C.C 1091202998



UNIVERSIDAD DEL QUINDÍO

Cristhian David Henao Hoyos

Universidad del Quindío
Facultad de Ingeniería
Ingeniería de sistemas y computación
Software III

Armenia, Quindío, 2026

Decisiones de Arquitectura (ADRs)

ADR-001: Lenguaje de Programación para el Backend

Estado	Aceptado
Contexto	Para desarrollar la API de *Huellitas Cafeteras* se necesita un lenguaje seguro, organizado y que todo el equipo domine bien.
Decisión	Se utilizará Java
Alternativas	1. Python 2. C#
Justificación	Es un lenguaje con el que ya se tiene experiencia, ofrece mucho control con el manejo de datos, gracias al fuerte tipado, y se integra perfecto con las herramientas de Spring Boot que se usaran.
Consecuencias	**Positivas:** Código tipado, mantenible y con amplio soporte técnico.
	Negativas: Requiere escribir mayor cantidad de código estructurado en comparación con lenguajes de tipado dinámico como Python o JavaScript

ADR-002: Definición del Estilo de Arquitectura del Sistema

Estado	Aceptado
Contexto	El proyecto necesita una estructura clara para separar la presentación, la lógica de negocio y el acceso a datos sin generar complejidad en el desarrollo del producto.
Decisión	Se adoptará un estilo de Arquitectura en Capas (Monolito Modular).
Alternativas	1. Arquitectura de Microservicios 2. Arquitectura Hexagonal
Justificación	La arquitectura en capas permite una separación de responsabilidades simples de mantener para el tamaño actual del equipo y del alcance del proyecto, sin añadir la complejidad de infraestructura, orquestación y latencia de red que conllevan, por ejemplo, los microservicios.
Consecuencias	**Positivas:** Desarrollo, pruebas y despliegue centralizados y sencillos.
	Negativas:

	Si el sistema crece de manera masiva, el escalamiento debe hacerse de forma global y no por componentes independientes.

ADR-003: Selección del Motor de Base de Datos

Estado	Aceptado
Contexto	Se requiere almacenar información estructurada de usuarios, reportes y bitácoras médicas garantizando consistencia y relaciones estrictas.
Decisión	Se utilizará la base de datos relacional PostgreSQL.
Alternativas	1. Oracle 2. MariaDB
Justificación	Garantiza la consistencia e integridad de los datos mediante el cumplimiento de propiedades ACID, permitiendo consultar la información de forma eficiente.
Consecuencias	**Positivas:** Datos estructurados seguros y sin duplicidad.
	Negativas: Menor flexibilidad si la estructura de los datos cambia drásticamente.

ADR-004: Selección del Protocolo de Comunicación

Estado	Aceptado
Contexto	El cliente web (Angular) y el servidor (Spring Boot) necesitan comunicarse de manera ligera, estandarizada y desacoplada.
Decisión	Se utilizará API RESTful mediante protocolo HTTP/HTTPS con formato JSON.
Alternativas	1. SOAP con formato XML
Justificación	REST sobre HTTP es el estándar más compatible, fácil de probar e integrar entre cliente y servidor.
Consecuencias	**Positivas:** Fácil consumo desde Angular y herramientas de pruebas de API.
	Negativa: Posible sobrecarga de peticiones en vistas que puedan ser complejas.

ADR-005: Estrategia de Autenticación de Usuarios

Estado	Aceptado
Contexto	Se requiere autenticar y controlar accesos de forma segura para los 4 roles del sistema, que serán: Ciudadano, Fundación, Veterinaria, Admin. Sin mantener sesión en servidor.
Decisión	Se implementará autenticación stateless con tokens JWT (JSON Web Token).
Alternativas	1. API Keys 2. Autenticación Básica HTTP 3. Autenticación con proveedores externos
Justificación	Permite validar roles e identidades en cada petición sin recargar la base de datos ni depender de memoria de sesión en el servidor.
Consecuencias	**Positiva:** Arquitectura desacoplada, ligera y escalable.
	Negativa: Requiere lógica adicional para invalidar tokens antes de su expiración.

ADR-006: Integración con Servicios Externos de Despliegue en Nube

Estado	Aceptado
Contexto	Se requiere publicar la plataforma en internet de forma automatizada para pruebas y demostración.
Decisión	Se integrará con Vercel para el frontend y Railway para el backend y base de datos.
Alternativas	1. AWS / Azure 2. Render / Netlify 3. Despliegue en un servidor local
Justificación	Facilita el despliegue continuo (CI/CD) directo desde el repositorio con costos nulos o mínimos y configuración rápida.
Consecuencias	**Positivas:** Publicación inmediata y mantenimiento de infraestructura automatizado.

	Negativas: Dependencia de los límites de consumo de los planes gratuitos.

ADR-007: Entorno de Desarrollo Local con Docker Compose

Estado	Aceptado
Contexto	Para evitar fallos por inconsistencias de versiones y dependencias entre las máquinas del equipo de desarrollo, se requiere estandarizar el entorno local para el backend y la base de datos.
Decisión	Se utilizará Docker Compose para orquestar los contenedores locales de PostgreSQL y la API en Java/Spring Boot.
Alternativas	1. Uso de máquinas virtuales (VMs).
Justificación	Docker Compose permite levantar los servicios de base de datos y backend de manera idéntica en cualquier equipo mediante un único archivo de configuración (docker-compose.yml), garantizando un entorno aislado y rápido de configurar.
Consecuencias	**Positivas:** Entorno de desarrollo unificado, rápida incorporación de desarrolladores y despliegue local replicable
	Negativas: Requiere consumo adicional de recursos de RAM/CPU en las máquinas locales para ejecutar el daemon de Docker.

ADR-008: Estrategia para Almacenamiento de Fotos y Archivos

Estado	Aceptado
Contexto	El registro de casos exige adjuntar fotografías del animal para su identificación, por tanto, guardar directamente estos archivos en la base de datos relacional puede afectar el rendimiento.
Decisión	Se utilizará un servicio de almacenamiento en la nube (*Cloudinary*), guardando únicamente la URL pública de referencia en PostgreSQL.
Alternativas	1. Amazon S3 2. Guardar archivos localmente en el servidor 3. Guardar en Base de Datos (BLOB)

Justificación	Almacenar sólo la URL optimiza el rendimiento de PostgreSQL, reduce el tamaño de las copias de seguridad y permite servir las imágenes de forma rápida a la interfaz en Angular.
Consecuencias	**Positivas:** Carga rápida de la plataforma y base de datos ligera.
	Negativas: Dependencia de un servicio externo para consultar las imágenes.

ADR-009: Selección de Herramientas para el Aseguramiento de Calidad de Código (SQA)

Estado	Aceptado
Contexto	Se requiere garantizar que el código fuente de Huellitas Cafeteras cumpla con los estándares de calidad, legibilidad, seguridad y mantenibilidad tanto en el backend como en el frontend, sin consumir la memoria ni los recursos de los servidores de producción/despliegue en Railway o Vercel
Decisión	Se utilizará **Checkstyle** para el backend (Java / Spring Boot), **ESLint** para el frontend (Angular / TypeScript) y **SonarCloud** (versión en la nube de SonarQube) integrado con **GitHub Actions** para el análisis estático continuo y la gestión automatizada de la deuda técnica.
Alternativas	1. PMD y FindBugs 2. SpotBugs y StyleCop 3. SonarLint como extensión independiente en IDE sin servidor centralizado
Justificación	Checkstyle y ESLint permiten verificar el cumplimiento de reglas y estándares de estilo durante el desarrollo local en Java y Angular. SonarCloud ofrece el mismo motor de análisis estático que SonarQube para medir seguridad, duplicación y cobertura de pruebas, pero con la ventaja de ser 100% gratuito para repositorios públicos de GitHub, automatizarse mediante GitHub Actions y no consumir recursos de infraestructura local o remota.
Consecuencias	**Positivas:** Cero costo de infraestructura para el servidor de calidad, análisis automático en cada Pull Request antes de hacer merge, estandarización de estilo en todo el equipo y prevención de vulnerabilidades de seguridad.
	Negativas: Dependencia de la disponibilidad del servicio externo de SonarCloud y necesidad de configurar secretos (SONAR_TOKEN) y workflows en GitHub Actions.

ADR-010: Estrategia de Pruebas y Cobertura de Código

Estado	Aceptado
Contexto	Se requiere garantizar que Huellitas Cafeteras cumpla un umbral mínimo de cobertura de pruebas del 60 %, tal como lo establece el objetivo de CI/CD del proyecto. Las herramientas definidas en el ADR-009 (Checkstyle, ESLint y SonarCloud) analizan la calidad del código, pero no ejecutan pruebas ni generan reportes de cobertura.
Decisión	Se utilizará JUnit 5 y Mockito para las pruebas unitarias y Spring Boot Test para las pruebas de integración en el backend, con JaCoCo para medir la cobertura. En el frontend se utilizará Vitest, el ejecutor de pruebas incluido por defecto en Angular, con su reporte de cobertura. Las pruebas se ejecutarán en cada Pull Request mediante GitHub Actions, y SonarCloud validará el umbral del 60 % mediante un Quality Gate.
Alternativas	1. TestNG y Cobertura 2. Jasmine y Karma en el frontend 3. Pruebas manuales sin medición automatizada
Justificación	JUnit 5, Mockito y Spring Boot Test son el estándar del ecosistema Spring Boot y se integran directamente con Maven. JaCoCo genera el reporte de cobertura que SonarCloud lee para calcular el porcentaje. Vitest viene configurado por defecto en las versiones recientes de Angular
Consecuencias	**Positivas:** Verificación automática del 60 % de cobertura en cada Pull Request, bloqueo del merge si no se cumple el umbral y detección temprana de errores.
	Negativas: Mayor tiempo de desarrollo para escribir las pruebas y mayor duración del pipeline en cada ejecución.

Modelo C4

Diagrama de Contexto del Sistema

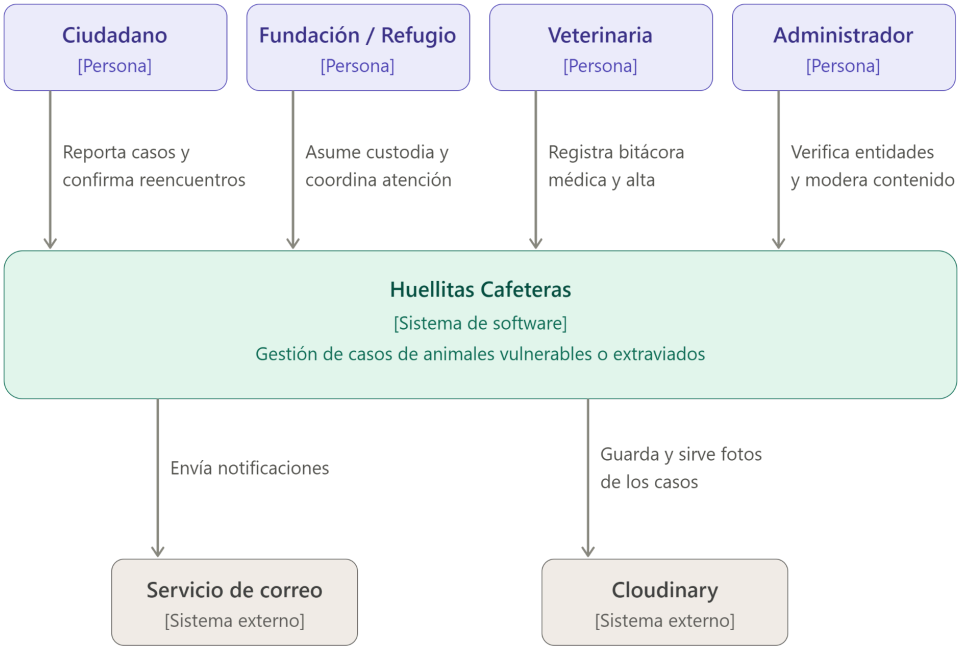
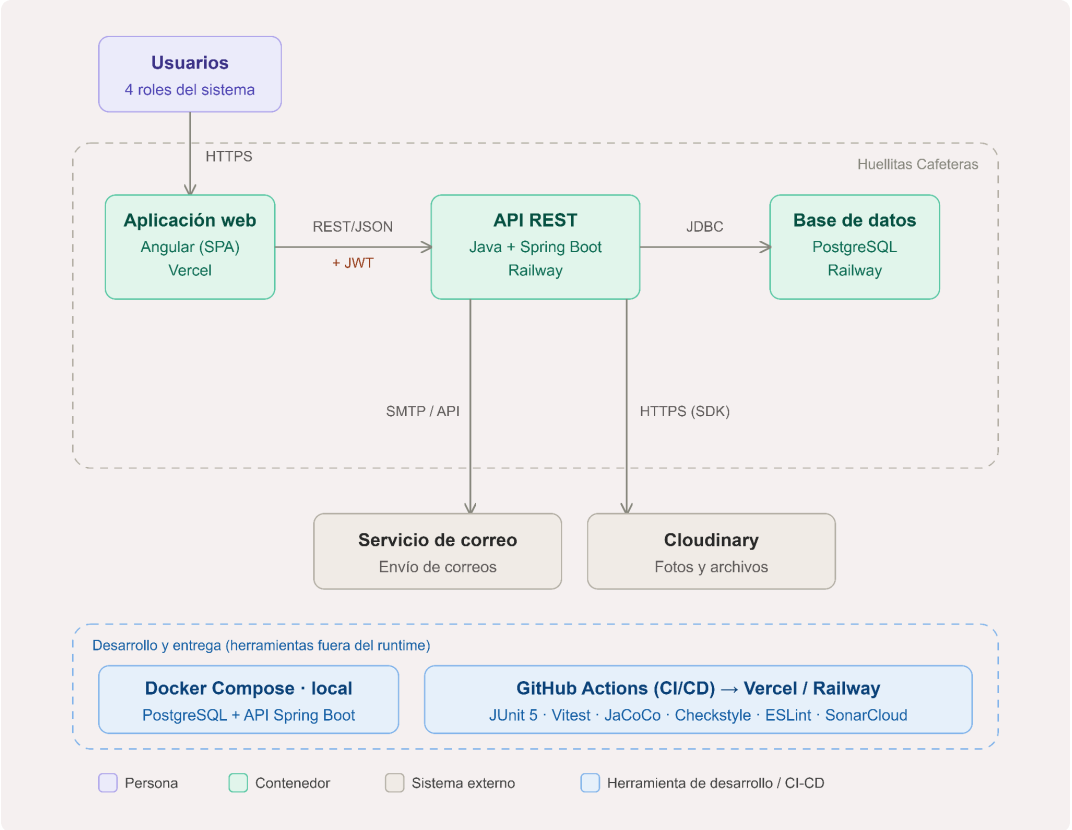


Diagrama de Contenedores



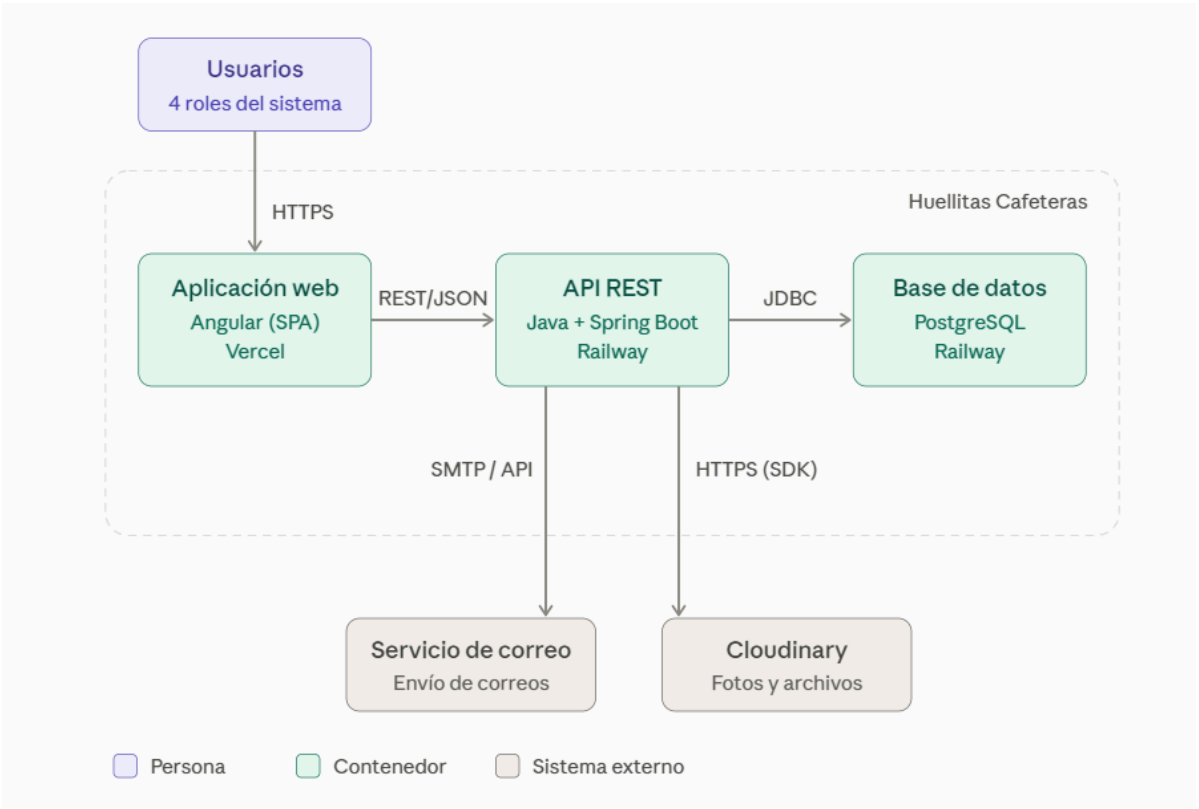


Diagrama de Componentes

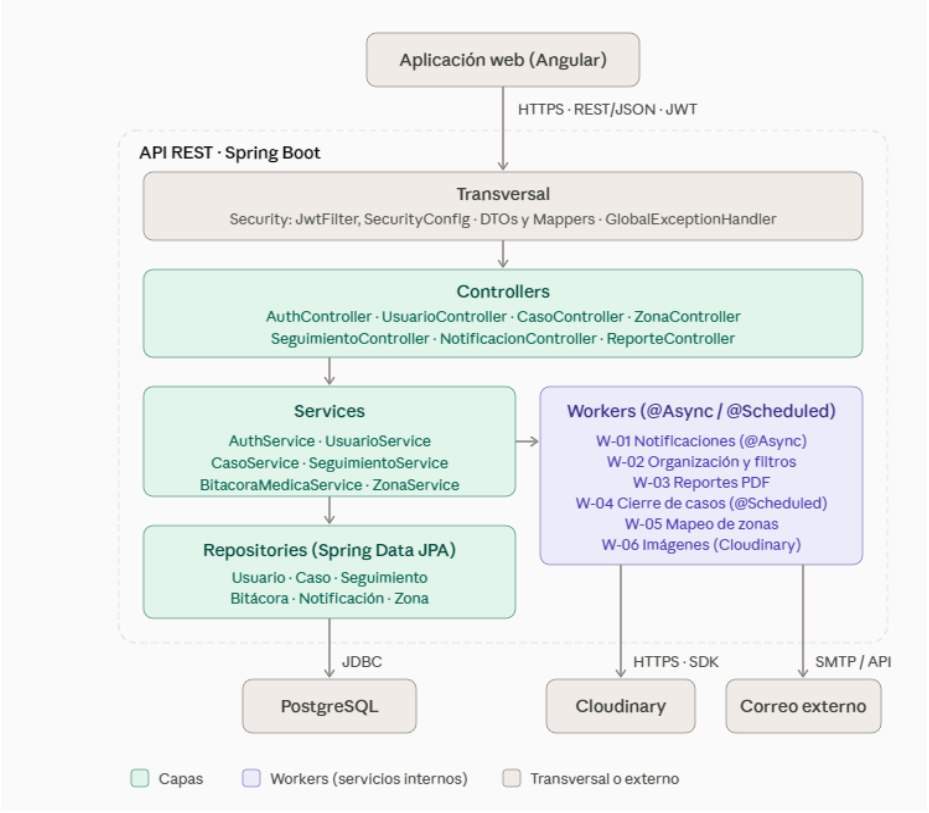
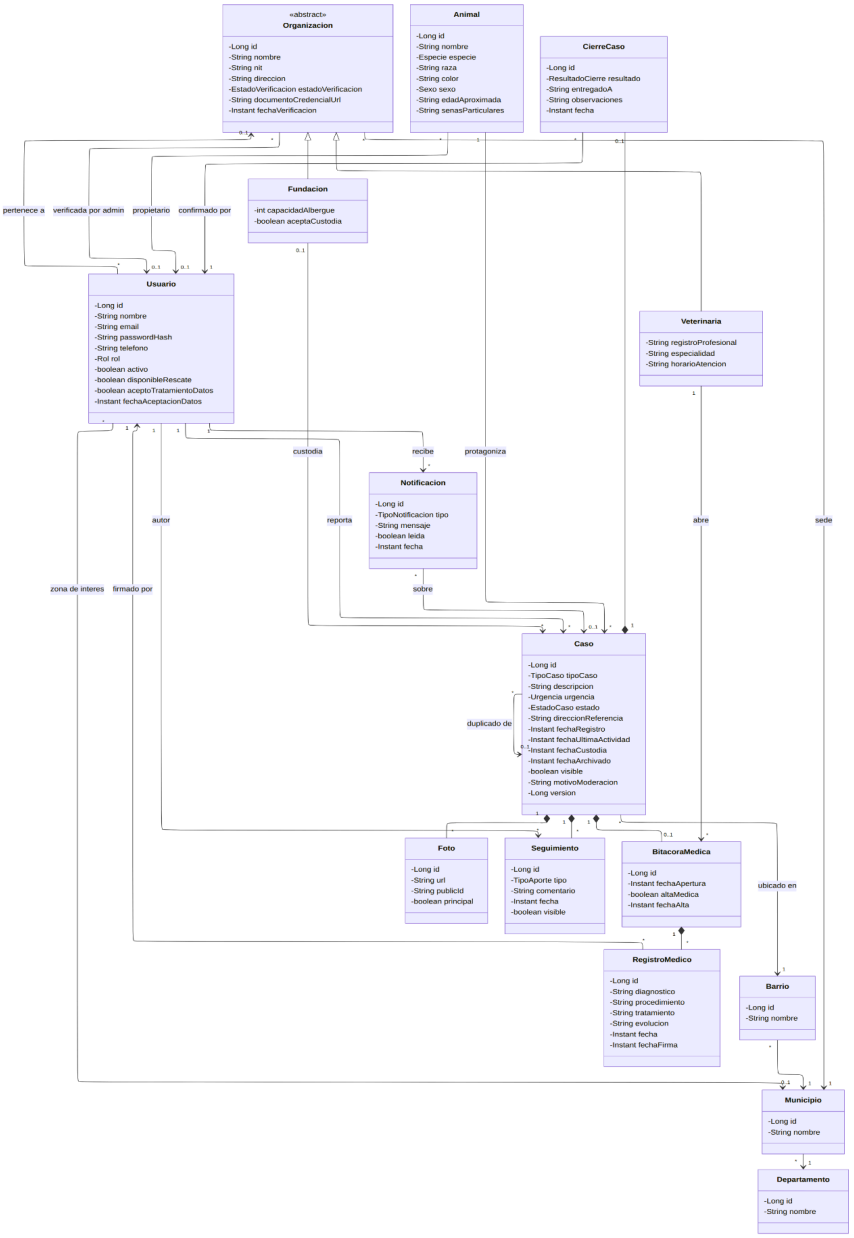


Diagrama de Código / Entidad-Relación

```
huellitas-cafeteras/
├── docker-compose.yml                          PostgreSQL + backend (desarrollo local · Docker Compose)
├── .github/workflows/                          CI/CD: pruebas, JaCoCo, Checkstyle, ESLint, SonarCloud
├── backend/                                    Spring Boot (Java) · Maven
│   ├── pom.xml                                 Spring Boot, JPA, JWT, JaCoCo, Checkstyle
│   ├── checkstyle.xml                          Reglas de estilo Java
│   ├── Dockerfile
│   └── src/
│       ├── main/java/com/huellitas/cafeteras/
│       │   ├── controller/                     Auth, Usuario, Caso, Seguimiento, ...
│       │   ├── service/                        Reglas de negocio
│       │   ├── worker/                         @Async y @Scheduled (W-01 a W-06)
│       │   ├── repository/                     Spring Data JPA
│       │   ├── entity/                         Usuario, Caso, Bitácora, Zona, ...
│       │   ├── dto/
│       │   ├── mapper/
│       │   ├── security/                       JwtFilter, SecurityConfig (JWT)
│       │   └── exception/                      GlobalExceptionHandler
│       ├── main/resources/                     application.yml, scripts SQL
│       └── test/java/                          JUnit 5 · Mockito · Spring Boot Test (cobertura JaCoCo)
└── frontend/                                   Angular (SPA)
    ├── package.json                            Dependencias, scripts de ESLint y Vitest
    ├── eslint.config.js                        Reglas de estilo TypeScript
    └── src/app/
        ├── core/                               Servicios HTTP, interceptor JWT, guards
        ├── shared/                             Componentes reutilizables
        └── features/
            ├── auth/
            ├── casos/                          Registro, feed y filtros
            ├── seguimiento/                    Bitácora y custodia
            └── admin/                          Verificación y moderación
```


«abstract»
Organizacion
-Long id
-String nombre
-String nit
-String direccion
-EstadoVerificacion estadoVerificacion
-String documentoCredencialUrl
-Instant fechaVerificacion
Animal
-Long id
-String nombre
-Especie especie
-String raza
-String color
-Sexo sexo
-String edadAproximada
-String senasParticulares
CierreCaso
-Long id
-ResultadoCierre resultado
-String entregadoA
-String observaciones
-Instant fecha
Fundacion
-int capacidadAlbergue
-boolean aceptaCustodia
Usuario
-Long id
-String nombre
-String email
-String passwordHash
-String telefono
-Rol rol
-boolean activo
-boolean disponibleRescate
-boolean aceptoTratamientoDatos
-Instant fechaAceptacionDatos
Veterinaria
-String registroProfesional
-String especialidad
-String horarioAtencion
Notificacion
-Long id
-TipoNotificacion tipo
-String mensaje
-boolean leida
-Instant fecha
Caso
-Long id
-TipoCaso tipoCaso
-String descripcion
-Urgencia urgencia
-EstadoCaso estado
-String direccionReferencia
-Instant fechaRegistro
-Instant fechaUltimaActividad
-Instant fechaCustodia
-Instant fechaArchivado
-boolean visible
-String motivoModeracion
-Long version
Foto
-Long id
-String url
-String publicId
-boolean principal
Seguimiento
-Long id
-TipoAporte tipo
-String comentario
-Instant fecha
-boolean visible
BitacoraMedica
-Long id
-Instant fechaApertura
-boolean altaMedica
-Instant fechaAlta
RegistroMedico
-Long id
-String diagnostico
-String procedimiento
-String tratamiento
-String evolucion
-Instant fecha
-Instant fechaFirma
Barrio
-Long id
-String nombre
Municipio
-Long id
-String nombre
Departamento
-Long id
-String nombre
pertenece a
verificada por admin
propietario
confirmado por
custodia
recibe
protagoniza
autor
reporta
abre
sede
zona de interes
firmado por
sobre
duplicado de
ubicado en